

Methodology: Predicting Chargebacks Before They Ship

Chargeback Prediction Model — Technical Appendix

Table of contents

1. Cross-retailer reason-code harmonization	1
2. Point-in-time feature engineering	2
3. Interpretable model design	4

Technical Appendix

This appendix documents three engineering choices that make the model interpretable, honest, and operationally useful: cross-retailer reason-code harmonization, point-in-time feature engineering, and a tree-based model with explicit SHAP attribution. Each addresses a failure mode that a naive implementation would miss.

1. Cross-retailer reason-code harmonization

The problem

Cinderhaven sells to five retailers. Each retailer uses its own vocabulary for chargeback reasons. One retailer charges a “label compliance fine.” Another charges a “barcode non-conformance.” A third uses “PLU error.” All three describe the same root cause: the product’s GTIN-14 barcode did not scan at the retailer’s DC.

A model trained on raw reason codes would treat these as three unrelated signals. It would fail to recognize that a spike in label compliance fines at one retailer is the same problem as a spike in barcode non-conformances at another.

The solution: dual-pathway harmonization

Two pathways map chargeback descriptions to five canonical archetypes:

Structured pathway. The Cinderhaven deduction codes table (`retailer_deduction_codes.deduction_type`) uses structured enum values. A lookup dict maps each code to exactly one archetype. An unmapped code raises an explicit error rather than silently falling through — this prevents new retailer codes from corrupting the model’s labels.

Unstructured pathway. The `retailer_chargebacks.reason` field is free text. An ordered keyword/regex list checks each description in priority order; the first match wins. Unmatched descriptions return `None` and are logged for manual review.

The five canonical archetypes

Archetype	Covers
<code>data_compliance_error</code>	Missing or invalid GTIN-14, UPC, or barcode at case level
<code>logistics_overage</code>	Case dimension or weight discrepancy; pallet configuration errors
<code>asn_timing_infraction</code>	ASN filed late or not at all relative to shipment
<code>pricing_discrepancy</code>	Invoice price differs from PO or promotional agreement
<code>item_setup_gap</code>	SKU not registered in retailer portal before first order
<code>legitimate</code>	Valid retailer deduction for confirmed over-shipment or spoilage

The mapping is defined in `src/harmonization/reason_codes.py`. Updating the mapping does not require touching the model — the harmonization layer is decoupled from the training pipeline.

2. Point-in-time feature engineering

Why today’s product master breaks the model

`product_master` is a snapshot of today’s data quality state. If a SKU’s GTIN-14 was missing in January 2024 but populated in March 2024, today’s `product_master` shows it as present. A model trained on today’s state would see a clean product master and have no way to know that the chargeback in February 2024 was caused by a GTIN-14 gap that no longer exists.

Training on today’s state leaks future information into historical labels. The model would learn from an impossible signal and produce predictions that cannot be reproduced on new data.

The solution: product_master_history

A purpose-built `product_master_history` table in the Cinderhaven Data Platform stores one row per SKU per monthly snapshot, covering the full shipment history date range. Each snapshot records the data quality state at that point in time:

```
sku | snapshot_date | gtin14_present | upc_present | case_dims_present | case_weight_present | da
```

For each shipment row in the training set, the feature join looks up the most recent snapshot where `snapshot_date` ≤ `ship_date`:

```
SELECT *
FROM product_master_history
WHERE sku = :sku AND snapshot_date <= :ship_date
ORDER BY snapshot_date DESC
LIMIT 1
```

This produces the data quality state at the moment the order shipped — not today’s state, not next month’s state. The distinction is the difference between a model that generalizes and one that trains on a fiction.

No-leakage chargeback rate

The `sku_prior_chargeback_rate` feature represents how often a SKU has been charged back historically. Computing it naively (using all rows in the dataset) would leak future chargebacks into past predictions. The feature is computed correctly using an expanding window: for each shipment row, only shipments strictly before the current `ship_date` contribute to the rate. Rows with no prior history use the dataset mean.

Feature set

Feature	Source	Notes
<code>gtin14_missing</code>	<code>product_master_history</code>	Point-in-time boolean
<code>upc_missing</code>	<code>product_master_history</code>	Point-in-time boolean
<code>case_dims_missing</code>	<code>product_master_history</code>	Point-in-time boolean
<code>case_weight_missing</code>	<code>product_master_history</code>	Point-in-time boolean
<code>data_quality_score</code>	<code>product_master_history</code>	Count of missing flags (0–4)
<code>asn_sent_late</code>	<code>retailer_shipments</code>	Boolean direct from source
<code>days_late</code>	<code>retailer_shipments</code>	Derived from ship vs. delivery date
<code>all_labels_scannable</code>	<code>retailer_shipments</code>	Boolean from shipment record
<code>sku_prior_chargeback_rate</code>	computed	Expanding window, no leakage
<code>retailer_id</code>	<code>retailer_orders</code>	One-hot encoded

3. Interpretable model design

The interpretability mandate

A probability score without an explanation is operationally useless. A buyer who sees “78% chargeback probability” cannot act on it without knowing *why*. A CFO evaluating the prevention roadmap cannot trust dollar figures derived from a black box.

The model is required to produce a plain-language attribution string for every prediction. No exception. The attribution string names the specific data condition driving the risk score and the archetype it maps to.

Model: GradientBoostingClassifier with SHAP

A GradientBoostingClassifier provides native SHAP (SHapley Additive exPlanations) support via `shap.TreeExplainer`. SHAP values have a precise interpretation: the SHAP value for feature f in row i represents the marginal contribution of f to the prediction for row i , relative to the mean prediction across the training set.

Positive SHAP values push the prediction above the mean (toward chargeback). Negative SHAP values push it below (away from chargeback). The feature with the largest positive SHAP value is the primary driver of the risk score.

The model uses `class_weight='balanced'` because chargebacks are rare events — approximately 1.5% of shipment-SKU rows result in a chargeback. Without class balancing, a naive model achieves high accuracy by predicting “no chargeback” for every row.

Temporal train/test split

The train/test split is strictly temporal, not random. Earlier shipments go to the training set; later shipments go to the test set. A random split would leak future shipments into training — the model would learn from data it could not have seen at prediction time and overstate its real-world performance.

Training set: 151,231 rows | Test set: 37,808 rows

Model performance

Metric	Value
AUC	0.6986
Precision	0.85
Recall	0.72
Training rows	151,231
Test rows	37,808

An AUC of **0.6986** means the model ranks a randomly chosen chargeback above a randomly chosen non-chargeback shipment 70% of the time — a *ranking* metric, not a flag rate. At the 0.5 threshold, recall of **0.72** is the share of actual chargebacks the model flags. Both are measured on held-out data the model did not train on, using a temporal split that mirrors the real scoring scenario.

What these numbers do — and do not — demonstrate

This must be stated plainly. On Cinderhaven’s **real** chargeback records, the upstream features carry no predictive signal: a baseline model scores AUC = 0.50 (indistinguishable from random) despite a 96.5% shipment-join rate. Cinderhaven’s synthetic database seeds chargebacks independently of data quality and ASN compliance, so no causal path exists to learn.

The metrics above are therefore measured on **synthetic training labels** generated by `scripts/generate_training_data.py` which draws each label from a known formula (chargeback risk = ASN lateness $\times$ data-quality gaps). The model’s job in this demonstration is to *recover that formula* — which it does, and the SHAP attribution and feature ordering match the formula’s multiplier structure. That is the honest scope of the result: it shows how the interpretable scoring architecture behaves against a correctly structured dataset. It is **not** evidence of predictive skill on Cinderhaven’s real chargebacks, where skill is effectively zero. The durable, real-data deliverable is the prevention roadmap, which is computed from the actual historical chargeback totals and fixed preventability fractions, independent of the model.

Attribution worked example

The following shows how a single high-risk shipment is scored and explained.

Input row:

Feature	Value
case_dims_missing	True
gtin14_missing	True
case_weight_missing	False
upc_missing	False
data_quality_score	2
asn_sent_late	False
days_late	1
all_labels_scannable	True
sku_prior_chargeback_rate	0.18
retailer_id	WMT

SHAP values (positive = pushes toward chargeback):

Feature	SHAP value
case_dims_missing	+0.31 ← top driver
sku_prior_chargeback_rate	+0.18
gtin14_missing	+0.12
data_quality_score	+0.09
asn_sent_late	+0.01
all_labels_scannable	-0.04
others	< ± 0.02

Model output: chargeback probability = **0.78**

Attribution string: > *Missing case dimensions → logistics audit → 78% probability within 14 days*

The attribution string names the top positive SHAP feature in plain language, the archetype it maps to, and the model’s probability estimate. This string is pre-computed at pipeline run time and embedded in the risk ledger JSON — no backend call required at display time.

All performance metrics are measured on a held-out temporal test set. The worked example uses synthetic Cinderhaven demonstration data. The attribution string template, SHAP computation, and model architecture described here are implemented in `src/pipeline/model.py`.